REPORTE DE PRÁCTICA NO. 3

NOMBRE DE LA PRÁCTICA: Construcción de un Autómata Finito Determinista

ALUMNO: Maria Fernanda Cortes Santos
Dr. Eduardo Cornejo-Velázquez

Introducción

Los Autómatas Finitos son máquinas teóricas que van cambiando de estado dependiendo de la entrada que reciben. La salida se limita a dos valores: aceptado y no aceptado, para indicar si la cadena que se ha recibido como entrada es o no válida.
Un Autómata Finito Determinista (AFD) se define como una tupla:

Un ***Autómata Finito Determinista (AFD)*** se define como una tupla:

$$\mathbf{AFD = (\Sigma,\ Q,\ \delta,\ q_0,\ F)}$$

- $\mathbf{\Sigma}$ es el alfabeto de entrada
- $\mathbf{Q}$ es el conjunto finito y no vacío de los estados
- $\boldsymbol{\delta}$ es la función de transición, $\boldsymbol{\delta}\mathbf{: Q \times \Sigma \rightarrow Q}$
- $q_0 \in Q$ es el estado inicial
- $\mathbf{F \subset Q}$ es un conjunto de estado finales de aceptación $(\mathbf{F \neq \emptyset})$

Sea $A = (\Sigma,\ Q,\ \delta,\ q_0,\ F)$ un AFD y sea $w = w_1 w_2 \dots w_n$ una cadena de símbolos donde $w_i \in \Sigma$.
Entonces, **A *acepta* w** si existe una secuencia de estados $r_0, r_1, \dots, r_n \in Q$ con tres condiciones:

1. $r_o = q_0$
2. $\delta(r_i, w_i + 1) = r_i + 1\ para\ i = 0,1, \dots, n-1$
3. $r_n \in F$

La condición 1 establece que la máquina comienza en el estado inicial. La condición 2 establece que la máquina cambia desde un estado hacia otro estado de acuerdo a la función de transición. La condición 3 establece que la máquina acepta la cadena de entrada si termina en un estado de aceptación. Entonces, A reconoce el lenguaje L si L = [w—A acepta w].

Marco teórico

El Determinismo en el Procesamiento de Cadenas

La característica fundamental de un AFD es la ausencia de ambigüedad. En términos de programación, esto implica que la función de transición f actúa como un algoritmo de ”estado siguiente” de tiempo constante. A diferencia de otros modelos, el AFD no requiere de backtracking (volver atrás); una vez que se lee un símbolo y se cambia de estado, el camino es definitivo.

Estructuras de Datos para la Función de Transición

Para implementar la tupla AFD=(Σ,Q,f,q0,F) en un lenguaje de alto nivel, la función de transición f:Q×Σ→Q se puede representar mediante dos estructuras principales:

- Matriz de Transición: Una tabla bidimensional donde las filas representan los estados (Q) y las columnas los símbolos del alfabeto (Σ). El valor en la celda [i,j] indica el estado destino. Es la forma más eficiente en memoria y velocidad de acceso.
- Listas de Adyacencia (Nodos y Objetos): Cada estado es un objeto que contiene un mapa o diccionario con las transiciones posibles. Es más flexible si el alfabeto es muy extenso.

Criterio de Aceptación

Una cadena W que pertenece un Σ es aceptada por el AFD si, al procesar la secuencia de símbolos de W partiendo del estado q0, el autómata termina en un estado qf que pertenece a F. En caso contrario, la cadena es rechazada.

Representación Computacional

Desde el punto de vista de la ingeniería de software, un AFD puede representarse como un grafo dirigido donde los estados son nodos y las transiciones son aristas etiquetadas. La implementación eficiente de la función f en lenguajes como C++ suele realizarse mediante tablas de dispersión (hash maps), las cuales permiten acceder a la transición destino en tiempo constante O(1), optimizando el rendimiento del simulador.

Objetivo General

Construir un programa en un lenguaje de programación de alto nivel para implementar un Autómata Finito Determinista (AFD) que permita analizar un conjunto de cadenas de símbolos (W) para determinar si pertenecen o no al lenguaje (L) reconocido por el AFD.

Objetivos Específicos

- Identificar los componentes de un Autómata Finito Determinista (AFD).
- Implementar un AFD en un lenguaje de programación de alto nivel.
- Determinar si diferentes conjunto de símbolos (W) pertenece al lenguaje (L) reconocido por el AFD.

Equipo y Software a utilizar

Equipo de cómputo con software para programación a elección de los alumnos.

Procedimiento para el desarrolo de la Práctica

Utilizar un lenguaje de programación de alto nivel para construir un programa que implemente un AFD con las siguientes condiciones:

Entrada

- La primera línea de entrada contiene seis enteros N, S, D, q_0, T y C ($1 \leq N, S, C \leq 100$, $1 \leq D \leq 100$, $1 \leq q_0 \leq N$, $0 \leq T \leq N$), donde $N = |Q|$, $S = |\Sigma|$, $D = N \times S$ es el número de transiciones en el autómata, q_0 es el estado inicial, $T = |F|$, y C es la cantidad de cadenas de símbolos a verificar si son aceptadas o no por el autómata M. Cada estado $q \in Q$ se identifica de manera implícita por un número entero con valor entre 1 y N.
- La segunda línea contiene el alfabeto Σ, representado por una secuencia de S símbolos separados por espacios, tal que cada símbolo puede ser una letra, un dígito o cualquier carácter del código ASCII excepto el espacio.
- La tercer línea contiene el conjunto de estados de aceptación F, representado por una secuencia de T enteros t_i ($1 \leq t_i \leq N$) separados por espacios.
- Las siguientes D líneas especifican las transiciones del autómata. Cada línea define una transición $\delta(I, X) = J$ por medio de un entero I, un carácter X y un entero J ($I, J \in Q$ y $X \in \Sigma$) separados por espacios, representando la transición desde el estado L hacia el estado J cuando el símbolo de la entrada sea X.
- Finalmente, cada una de las siguientes C líneas contienen una cadena W, compuesta por símbolos que pertenecen al alfabeto Σ. La longitud de la cadena W está entre 0 y 100 caracteres.

Salida

- Para cada una de las C cadenas W se deberá imprimir el mensaje ACEPTADA si el autómata M acepta la cadena W, ó RECHAZADA en caso contrario.

Ejemplo

La siguiente figura es la representación gráfica de una AFD que reconoce palabras que tienen una cantidad par de 0's después del último 1.

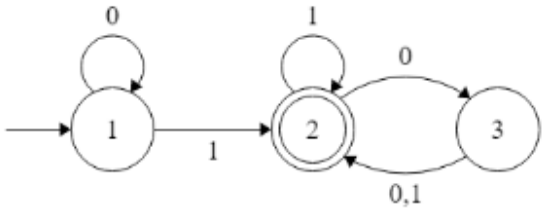

Figura 1. AFD.

El autómata tiene N = 3 estados (Q = {1, 2, 3}), un alfabeto S = 2 símbolos (Σ = {0, 1}), se definen D = 6 transiciones, el estado inicial es q_0 = 1 y el estado final de aceptación T = 1 (F = {2}).
Con base en lo anterior, las entradas al programa para verificar C = 3 palabras son:

```
3 2 6 1 1 4
0 1
2
1 0 1
1 1 2
2 0 3
2 1 2
3 0 2
3 1 2
101
0
11
1000100001
```

El ejemplo verifica C = 3 palabras W (líneas 10−12). Con los siguientes resultados:

101	ACEPTADA
0	RECHAZADA
11	ACEPTADA
1000100001	ACEPTADA

Resultados (código)

https://github.com/MaFecs/Actividades_Automatas_y_Compiladores/blob/c0e313ba894491c30a320ac4d748dc765cd
ConstruccionAFD.cpp

Cuestionario

- ¿Qué paradigma de programación utilizaste para la implementación del AFD? Se utilizó el paradigma de programación imperativa, específicamente un enfoque estructurado. El programa se basa en una secuencia explícita de instrucciones que modifican el estado del sistema (variables como estado-actual o el contenido de los contenedores) paso a paso hasta llegar a un resultado final.

- ¿Cuáles elementos del lenguaje de programación utilizaste?
  - Contenedores de la STL (Standard Template Library): vector, unordered-map y unordered-set.
  - Control de flujo: Estructuras iterativas (for) y condicionales (if-else, break) para la navegación del autómata.
  - Entrada/Salida Estándar: Uso de cin para la configuración dinámica y cout para presentar los resultados.
  - Tipos de datos: Tipos primitivos como int, char, bool y tipos complejos como string.
- ¿Qué estructuras de datos utilizaste para implementar los componentes del AFD? Para representar el autómata de forma eficiente y dinámica se utilizaron:
  - vector¡unordered-map¡char, int¿¿: Para la función de transición (). El vector funciona como una tabla indexada por estados, mientras que cada unordered-map actúa como un diccionario que asocia el símbolo de entrada con el estado destino.
  - unordered-set¡int¿: Para representar el conjunto de estados de aceptación (F). Proporciona una forma de verificar la pertenencia de un estado al conjunto de estados finales con una complejidad de tiempo O(1).
  - vector¡string¿: Para el almacenamiento temporal de la lista de cadenas (W) a evaluar.
- ¿Qué elementos del lenguaje de programación te permitieron implementar las funciones de transición? Implementación: La combinación de manual-de-reglas[estado-actual].count(letra) (para verificar existencia) y manual-de-reglas[estado-actual][letra] (para recuperar el siguiente estado) permitió implementar f de manera determinista y eficiente.
- ¿cuáles son las limitaciones del programa que construiste?
  - Memoria: Aunque es dinámica, el uso de tablas hash implica un mayor consumo de memoria en comparación con matrices estáticas para autómatas muy densos.
  - Validación de entrada: El programa asume que la entrada sigue estrictamente el formato especificado; no incluye un manejo robusto de errores si el usuario ingresa datos que no coinciden con la tupla esperada (por ejemplo, estados fuera del rango 1 a N).
- ¿Qué condiciones se deben cumplir para que el programa pueda evaluar cualquier AFD? Para que el simulador sea universal, se deben cumplir los siguientes requisitos en los datos de entrada:
  - Consistencia de la Tupla: Se deben proporcionar los seis parámetros iniciales (N,S,D,q0,T,C) correctamente.
  - Definición de transiciones: Deben existir exactamente D líneas de transición que cubran la lógica del autómata.
  - Alfabeto coherente: Las cadenas a evaluar deben estar compuestas únicamente por los símbolos definidos previamente en el alfabeto Σ.
  - Estados válidos: Los identificadores de estados utilizados en las transiciones deben estar dentro del rango [1,N] definido inicialmente.

Conclusiones

La implementación de este AFD demostró que el diseño de algoritmos eficientes no depende solo de la lógica matemática, sino de la selección estratégica de estructuras de datos. Investigar y aplicar herramientas como unordered-set, unordered-map y vector fue fundamental para alcanzar una complejidad lineal O(N) y asegurar que el sistema procesara las cadenas con la rapidez requerida. El dominio de estas estructuras permite transformar definiciones teóricas en soluciones prácticas optimizadas. Este proyecto confirma que, al combinar estrategias de "divide y vencerás" con estructuras de la biblioteca estándar de C++, es posible construir un simulador configurable, eficiente y escalable para el reconocimiento de lenguajes regulares, tal como lo validan los resultados obtenidos en la plataforma omegaUp. En conclusión, la eficiencia algorítmica es el resultado directo de una implementación técnica bien fundamentada.

Referencias Bibliográficas

- TylerMSFT. (s.f.). unordered-set (Clase). Microsoft Learn. https://learn.microsoft.com/es-es/cpp/standard-library/unordered-set-class?view=msvc-170
- Hopcroft, J. E., Motwani, R., y Ullman, J. D. (2007). Introduction to Automata Theory, Languages, and Computation (3.ª ed.). Pearson Education.
- Sipser, M. (2012). Introduction to the Theory of Computation (3.ª ed.). Cengage Learning.
- Torres-Pérez, A., Cornejo-Velázquez, E., y Clavel-Maqueda, M. (2026). Design of a recursive algorithm for DFA implementation: computational complexity analysis. International Journal Of Combinatorial Optimization Problems And Informatics., 17(1), 299-306. https://doi.org/10.61467/2007.1558.2026.v17i1.1172